

# Flipkart Brand Onboarding

## Gherkin Test Cases

|         |                                                  |
|---------|--------------------------------------------------|
| Author  | QA Lead                                          |
| Version | 1.1                                              |
| Date    | 2026-03-18                                       |
| PRD     | 2026-03-16-flipkart-brand-login-signup-design.md |
| Format  | Gherkin BDD                                      |

Tags: @smoke @functional @negative @boundary @edge @security @concurrency @e2e @integration @data-integrity @db  
@email @unit @state @api @accessibility @ux



## 1. FE — Login Page

**Feature: Flipkart Brand Login Page (/flipkart/login)**

**Background:**

**Given** the user navigates to "/flipkart/login"

# — *Functional* —

@smoke @functional

**Scenario: Page renders with correct elements**

**Then** the page title or heading contains "Flipkart"

**And** a "Login with Flipkart" button is visible

**And** a "Don't have an account? Sign up" link is visible

@smoke @functional

**Scenario: Successful OAuth login for an onboarded user**

**Given** the user is not logged in

**When** the user clicks "Login with Flipkart"

**And** the Flipkart OAuth popup opens successfully

**And** the user completes authentication

**And** the backend returns "{ onboarded: true, token: '<jwt>' }"

**Then** the JWT is stored in localStorage

**And** the user is redirected to "/home"

@smoke @functional

**Scenario: Successful OAuth for a new (not onboarded) user**

**Given** the user is not logged in

**When** the user clicks "Login with Flipkart"

**And** the user completes Flipkart OAuth

**And** the backend returns "{ onboarded: false }"

**Then** the user is redirected to "/flipkart/signup?code=<auth\_code>"

**And** no JWT is stored in localStorage

@functional

**Scenario: "Sign up" footer link navigates correctly**

**When** the user clicks "Don't have an account? Sign up"

**Then** the user is navigated to "/flipkart/signup"

# — *Session / State* —

@functional @state

**Scenario: Already logged-in user is redirected away**

**Given** the user has a valid JWT stored in localStorage

**When** the user navigates to "/flipkart/login"

**Then** the user is immediately redirected to "/home"

**And** the login page is never rendered

@functional @state

**Scenario: Expired JWT does not redirect – re-login required**

**Given** the user has an expired JWT stored in localStorage  
**When** the user navigates to "/flipkart/login"  
**Then** the login page is rendered normally  
**And** the user is not redirected

# — Negative —————

@negative

**Scenario: OAuth popup is blocked by the browser**

**Given** the browser is configured to block popups  
**When** the user clicks "Login with Flipkart"  
**Then** a message "Please allow popups for this site" is displayed  
**And** a "Try again" or "Enable popups" call-to-action is shown

@negative

**Scenario: User closes the OAuth popup without completing login**

**When** the user clicks "Login with Flipkart"  
**And** the OAuth popup opens  
**And** the user closes the popup without authenticating  
**Then** no redirect occurs  
**And** the login page remains visible  
**And** no error banner is shown

@negative

**Scenario: Flipkart OAuth service is unavailable**

**Given** the Flipkart OAuth service is down  
**When** the user clicks "Login with Flipkart"  
**Then** an error message is displayed  
**And** a "Try again" button is shown

@negative

**Scenario: SSO API call fails with server error**

**Given** the OAuth completes successfully returning an auth code  
**When** the frontend calls "POST /api/auth/flipkart-sso"  
**And** the API responds with HTTP 500  
**Then** an error banner is shown  
**And** a "Try again" button is visible  
**And** the user is not redirected

@negative

**Scenario: SSO API call returns invalid auth code error**

**Given** the OAuth completes returning an expired auth code  
**When** the frontend calls "POST /api/auth/flipkart-sso"  
**And** the API responds with HTTP 400  
**Then** an appropriate error message is displayed

# — Edge Cases —————

@edge

**Scenario: Network drops during SSO API call**

**Given** the OAuth completes successfully  
**When** the frontend calls "POST /api/auth/flipkart-sso"  
**And** the network connection is lost mid-request  
**Then** a network error message is shown  
**And** a "Retry" option is available

@edge

**Scenario: User double-clicks "Login with Flipkart"**

**When** the user double-clicks the "Login with Flipkart" button  
**Then** only one OAuth popup is opened  
**And** duplicate API calls are prevented

@edge

**Scenario: Page is loaded on a mobile device**

**Given** the user is on a mobile browser  
**When** the user navigates to "/flipkart/login"  
**Then** the page is fully responsive and all elements are visible

# — Security —————

@security

**Scenario: JWT is not exposed in the URL**

**Given** the SSO returns a valid JWT  
**When** the user is redirected to "/home"  
**Then** the JWT does not appear in the URL query parameters

@security

**Scenario: Login page is served over HTTPS**

**When** the user navigates to "/flipkart/login"  
**Then** the page is loaded over a secure HTTPS connection

@security

**Scenario: Auth code in redirect URL is consumed and not reusable**

**Given** the user is redirected to "/flipkart/signup?code=AUTH\_CODE"  
**When** the auth code "AUTH\_CODE" is used to complete signup  
**And** the same auth code is submitted again to "POST /api/auth/flipkart-sso"  
**Then** the second request returns HTTP 400

## 2. FE — Signup Step 1: Connect Accounts

**Feature: Flipkart Brand Signup - Step 1: Connect Accounts**

**Background:**

**Given** the user is on `"/flipkart/signup"`

**And** the stepper shows Step 1 as active

# — *Functional* —————

@smoke @functional

**Scenario: Accounts auto-loaded when auth code is in URL**

**Given** the URL contains `"?code=VALID_AUTH_CODE"`

**When** the page loads

**Then** the auth code is automatically exchanged

**And** a list of Flipkart ad accounts is displayed as checkboxes

@functional

**Scenario: Show OAuth button when no auth code in URL**

**Given** the URL does not contain a `"?code="` parameter

**When** the page loads

**Then** a "Login with Flipkart" button is shown

**And** no account list is displayed yet

@functional

**Scenario: User selects one account and proceeds**

**Given** the account list is displayed with 3 accounts

**When** the user selects 1 account via checkbox

**Then** the "Next" button becomes enabled

**When** the user clicks "Next"

**Then** the wizard advances to Step 2

@functional

**Scenario: User selects multiple accounts**

**Given** the account list shows 5 ad accounts

**When** the user selects all 5 accounts

**Then** all 5 are shown as checked

**And** the "Next" button is enabled

# — *Negative* —————

@negative

**Scenario: User attempts to proceed with no accounts selected**

**Given** the account list is displayed

**When** the user has selected 0 accounts

**Then** the "Next" button is disabled

Or a validation message "Please select at least one account" is shown

@negative

**Scenario: Auth code exchange fails (invalid/expired code)**

**Given** the URL contains "?code=INVALID\_CODE"  
**When** the page tries to exchange the code  
**Then** an error message is displayed  
**And** a "Try again" button is shown  
**And** no account list is rendered

@negative

**Scenario: Flipkart returns zero ad accounts for the user**

**Given** the auth code is valid  
**But** the Flipkart API returns an empty ad accounts list  
**Then** a message "No ad accounts found for your Flipkart profile" is shown  
**And** the "Next" button is disabled

# — Boundary —————

@boundary

**Scenario: Account list with exactly 1 account**

**Given** the Flipkart API returns exactly 1 ad account  
**Then** that 1 account is shown with a checkbox  
**And** the user can select it and proceed

@boundary

**Scenario: Account list with a large number of accounts (virtualized list)**

**Given** the Flipkart API returns 200 ad accounts  
**Then** the accounts are rendered using the virtualized list component  
**And** the page does not freeze or crash  
**And** scrolling through the list works correctly

# — Edge Cases —————

@edge

**Scenario: Ad account names contain special characters**

**Given** one ad account has the name "Brand & Co. <Test> 'Pvt' Ltd"  
**Then** the account name is displayed correctly without XSS rendering

@edge

**Scenario: Ad account names contain Unicode characters**

**Given** one ad account name is in Hindi or another non-Latin script  
**Then** the name is displayed correctly

@edge

**Scenario: Auth code expires mid-session (after accounts are loaded)**

**Given** the accounts are already displayed from a valid auth code  
**And** the auth code is stored in React state  
**When** the user takes a long time to complete the wizard  
**And** the Flipkart auth code TTL expires before "POST /api/flipkart/signup" is called  
**Then** the backend's exchangeAuthCodeForTokens call fails  
**And** the user sees a generic error on Step 4 submission  
**And** no partial records are created

**And** the user must restart the signup flow from the beginning  
**And** no proactive expiry detection or auto-reauth is performed during the wizard

@edge

**Scenario: User navigates back from Step 2 to Step 1**

**Given** the user has already selected 2 accounts and reached Step 2  
**When** the user clicks the "Back" button  
**Then** Step 1 is shown  
**And** the previously selected 2 accounts are still checked

@edge

**Scenario: Same ad account connected to another company**

**Given** one of the returned ad accounts is already linked to another company  
**When** the account list is displayed  
**Then** no warning indicator is shown on the FE (FE renders each account as a unique checkbox)  
**And** the user can select and proceed normally  
**And** on the backend, the upsert for connected\_accounts silently merges the duplicate  
**And** no error is returned and no data corruption occurs



### 3. FE — Signup Step 2: Agency Code

**Feature: Flipkart Brand Signup - Step 2: Agency Code**

**Background:**

**Given** the user has completed Step 1 and is on Step 2

# — *Functional* —————

@smoke @functional

**Scenario: Valid agency code shows agency name with confirmation**

**When** the user types "ATR4X2" in the agency code field  
**And** the API "GET /api/agency/validate-code/ATR4X2" returns valid  
**Then** a green checkmark is displayed  
**And** the agency name (e.g., "MediaCom Agency") is shown below the input  
**And** the "Next" button becomes enabled

@functional

**Scenario: Agency code is normalized on input — both FE and BE**

**When** the user types "atr4x2" in lowercase  
**Then** the FE normalizes it: `toUpperCase().replace(/^[^A-Z0-9]/g, "").slice(0, 6)`  
**And** the input field displays "ATR4X2"  
**And** the validation API is called with "ATR4X2"  
**And** the BE also normalizes via `code.trim().toUpperCase()` before DB lookup

@functional

**Scenario: Agency code auto-filled from URL query param**

**Given** the URL contains "?agency\_code=ATR4X2"  
**When** Step 2 loads  
**Then** the agency code field is pre-filled with "ATR4X2"  
**And** validation is automatically triggered  
**And** the agency name is shown if the code is valid

@functional

**Scenario: "I don't have a code" link toggles inline info box**

**When** the user clicks "I don't have a code"  
**Then** a gray info box is revealed inline below the agency code input  
**And** it displays: "An agency code is provided by your advertising agency. If you don't have one, please contact your agency representative or reach out to our support team for assistance."  
**And** no modal or new page is opened  
**When** the user clicks the link again  
**Then** the info box is hidden (toggle behaviour)

@functional

**Scenario: Back button preserves Step 1 selections**

**When** the user clicks "Back"  
**Then** Step 1 is displayed  
**And** previously selected accounts are still checked

# — *Negative* —————

@negative

**Scenario: Invalid agency code shows inline error**

**When** the user types "XXXXXX"  
**And** the API returns "{ valid: false }"  
**Then** an inline error "Invalid agency code" is displayed  
**And** no agency name is shown  
**And** the "Next" button remains disabled

@negative

**Scenario: Agency code longer than 6 characters is rejected by input**

**When** the user attempts to type "ATR4X2ZZ" (8 chars)  
**Then** only "ATR4X2" (first 6 chars) is accepted  
**And** the extra characters are not entered

@negative

**Scenario: User tries to proceed with empty agency code field**

**Given** the agency code field is empty  
**When** the user clicks "Next"  
**Then** a validation error is shown  
**And** the wizard does not advance

@negative

**Scenario: Agency code validation API call fails (network error)**

**When** the user types a code  
**And** the validation API call times out  
**Then** an error "Unable to validate code, please try again" is shown  
**And** the "Next" button remains disabled

# — Boundary —————

@boundary

**Scenario Outline: Agency code length boundary validation**

**When** the user types "<code>" in the agency code field  
**Then** the UI enforces a maximum of 6 characters  
**And** the field contains "<accepted>"

**Examples:**

|           |          |
|-----------|----------|
| code      | accepted |
| A         | A        |
| ATR4      | ATR4     |
| ATR4X2    | ATR4X2   |
| ATR4X2Z   | ATR4X2   |
| ATR4X2ZZZ | ATR4X2   |

@boundary

**Scenario: Exactly 6 character valid code passes validation**

**When** the user types exactly "ATR4X2"  
**And** the API confirms it is valid  
**Then** the validation succeeds and "Next" is enabled

@boundary

**Scenario: Exactly 5 characters – Next button stays disabled**

**When** the user types "ATR4X" (5 chars)  
**Then** the "Next" button is disabled  
**And** no API validation call is made until 6 chars are entered

# — Edge Cases —————

@edge

**Scenario: Agency code with special characters is stripped by FE normalization**

**When** the user attempts to type "ATR-4!"  
**Then** the FE normalization strips non-alphanumeric characters via `replace(/^[A-Z0-9]/g, "")`  
**And** the input field shows only "ATR4"  
**And** if the remaining value is less than 6 chars, the "Next" button stays disabled

@edge

**Scenario: User pastes a code from clipboard**

**When** the user pastes " atr4x2 " (with spaces) from clipboard  
**Then** the value is trimmed and uppercased to "ATR4X2"  
**And** validation is triggered

@edge

**Scenario: Agency code validation is debounced**

**When** the user types characters rapidly one by one  
**Then** the API is not called on every keystroke  
**And** only one API call is made after the user stops typing

@edge

**Scenario: Agency code belongs to a non-agency company**

**Given** a code exists in the database but belongs to a client company  
**When** the user enters that code  
**Then** the API returns "{ valid: false }"  
**And** an inline error is shown

@edge

**Scenario: Auto-filled agency code from URL is invalid**

**Given** the URL contains "?agency\_code=BADCOD"  
**When** Step 2 loads and auto-validates  
**Then** an inline error "Invalid agency code" is shown automatically  
**And** the field is still editable for correction

@edge

**Scenario: User returns to Step 2 after navigating forward – code preserved**

**Given** the user entered a valid code "ATR4X2" on Step 2  
**And** the user proceeded to Step 3  
**When** the user clicks "Back" to return to Step 2  
**Then** the code "ATR4X2" is still displayed  
**And** the green checkmark and agency name are still shown



## 4. FE — Signup Step 3: Agency Permissions

**Feature: Flipkart Brand Signup - Step 3: Agency Permissions**

**Background:**

**Given** the user has completed Steps 1 and 2 and is on Step 3

# — *Functional* —————

@smoke @functional

**Scenario: Both permission options are displayed**

**Then** a "Read Only" card/option is visible

**And** a "Read & Write" card/option is visible

**And** each option shows a description of its access level

@functional

**Scenario: User selects "Read Only" and proceeds**

**When** the user clicks the "Read Only" card

**Then** it is visually selected (highlighted/checked)

**And** the "Next" button becomes enabled

**When** the user clicks "Next"

**Then** the wizard advances to Step 4

@functional

**Scenario: User selects "Read & Write" and proceeds**

**When** the user clicks the "Read & Write" card

**Then** it is visually selected

**And** the "Next" button becomes enabled

**When** the user clicks "Next"

**Then** the wizard advances to Step 4

@functional

**Scenario: User can switch between options before proceeding**

**When** the user clicks "Read Only"

**Then** "Read Only" is selected

**When** the user then clicks "Read & Write"

**Then** "Read & Write" is selected

**And** "Read Only" is deselected

@functional

**Scenario: Description of Read Only permission is accurate**

**When** the user views the "Read Only" description

**Then** it mentions view-only access to reports, campaigns, budgets

**And** mentions no ability to edit or create

@functional

**Scenario: Description of Read & Write permission is accurate**

**When** the user views the "Read & Write" description

**Then** it mentions ability to create/edit campaigns, budgets, tags

**And** mentions inability to disconnect accounts or delete workspace

# — *Negative* —————

@negative

**Scenario: User tries to proceed without selecting a permission**

**Given** no permission option is selected

**When** the user clicks "Next"

**Then** the wizard does not advance

**And** a validation message "Please select a permission level" is shown  
Or the "Next" button is disabled until a selection is made

# — *State* —————

@state

**Scenario: Permission selection is preserved when navigating back and forward**

**When** the user selects "Read & Write"

**And** proceeds to Step 4

**And** then clicks "Back"

**Then** "Read & Write" is still selected on Step 3

@state

**Scenario: Default state — no pre-selection on first visit**

**When** the user arrives on Step 3 for the first time

**Then** neither option is pre-selected

**And** the "Next" button is disabled

## 5. FE — Signup Step 4: Invite Team

**Feature: Flipkart Brand Signup - Step 4: Invite Team (Optional)**

### Background:

**Given** the user has completed Steps 1-3 and is on Step 4

# — Functional —————

@smoke @functional

### Scenario: User skips team invitation

**When** the user clicks "Skip"  
**Then** "POST /api/flipkart/signup" is called without "invited\_emails"  
**And** signup completes successfully  
**And** the user is redirected to "/home"

@functional

### Scenario: User adds a valid email and finishes

**When** the user types "teammate@brand.com" in the email input  
**And** clicks "Add"  
**Then** "teammate@brand.com" appears in the invite list  
**When** the user clicks "Finish"  
**Then** "POST /api/flipkart/signup" is called with "invited\_emails: ['teammate@brand.com']"

@functional

### Scenario: User adds multiple valid emails

**When** the user adds "a@brand.com", "b@brand.com", and "c@brand.com"  
**Then** all 3 emails appear in the invite list  
**When** the user clicks "Finish"  
**Then** the API is called with all 3 emails in "invited\_emails"

@functional

### Scenario: User removes an email from the invite list

**Given** the user has added "a@brand.com" to the invite list  
**When** the user clicks the remove/delete icon next to "a@brand.com"  
**Then** "a@brand.com" is removed from the list  
**And** the invite list shows 0 emails

@functional

### Scenario: "Finish" is available even with an empty invite list

**Given** the invite list is empty  
**Then** the "Finish" button is visible and enabled  
**And** clicking "Finish" calls the API without "invited\_emails"

# — Negative —————

@negative

### Scenario: User tries to add an invalid email format

**When** the user types "notanemail" and clicks "Add"

**Then** a validation error "Please enter a valid email address" is shown  
**And** "notanemail" is not added to the list

@negative

#### Scenario Outline: Various invalid email formats are rejected

**When** the user types "<invalid\_email>" and clicks "Add"  
**Then** a validation error is shown  
**And** the email is not added

##### Examples:

|                    |  |
|--------------------|--|
| invalid_email      |  |
| notanemail         |  |
| @nodomain.com      |  |
| user@              |  |
| user@.com          |  |
| user name@test.com |  |
| user@@test.com     |  |

@negative

#### Scenario: User attempts to add a duplicate email

**Given** "a@brand.com" is already in the invite list  
**When** the user types "a@brand.com" again and clicks "Add"  
**Then** an error "Email already added" is shown  
**And** the list still contains only one "a@brand.com"

@edge

#### Scenario: Brand owner adds their own email to the invite list

**Given** the brand owner's email is "owner@brand.com" (from the OAuth token)  
**When** the user types "owner@brand.com" and clicks "Add"  
**Then** the FE does not prevent this and adds the email to the list  
**When** the user clicks "Finish" with "owner@brand.com" in invited\_emails  
**Then** the invitation email is sent to "owner@brand.com"  
**But** no duplicate user record is created (the existing user check handles this at login)

# — Boundary —————

@boundary

#### Scenario: Very long email address

**When** the user types an email with 254 characters (RFC 5321 max) and clicks "Add"  
**Then** it is accepted if the format is valid

@boundary

#### Scenario: Email with "+" alias is accepted

**When** the user types "user+alias@example.com" and clicks "Add"  
**Then** it is added to the invite list without error

@boundary

#### Scenario: Adding exactly 1 email — minimum invite

**When** the user adds 1 valid email  
**Then** the "Finish" button calls the API with 1 email in "invited\_emails"



@edge

**Scenario: Pressing Enter key adds email instead of clicking Add**

**When** the user types "user@test.com" and presses the Enter key

**Then** "user@test.com" is added to the invite list

@edge

**Scenario: Email input is trimmed of leading/trailing spaces**

**When** the user types " user@test.com " with spaces

**And** clicks "Add"

**Then** "user@test.com" (trimmed) is added to the list

@edge

**Scenario: User adds email, navigates back, and returns to Step 4**

**Given** the user has added "a@brand.com"

**When** the user clicks "Back" to go to Step 3

**And** then clicks "Next" to return to Step 4

**Then** "a@brand.com" is still in the invite list

@edge

**Scenario: Large number of invite emails**

**When** the user adds 50 valid emails to the invite list

**Then** all 50 are displayed (possibly with scroll)

**And** the page does not freeze

**And** "Finish" sends all 50 in "invited\_emails"

## 6. FE — Signup Complete Flow & Wizard Navigation

### Feature: Flipkart Brand Signup - Complete Flow and Wizard Navigation

# — Happy Path —

@smoke @functional @e2e

#### Scenario: Full happy path — new brand completes all 4 steps

**Given** the user is on "/flipkart/signup?code=VALID\_CODE&agency\_code=ATR4X2"  
**When** Step 1 loads with ad accounts auto-fetched  
**And** the user selects 2 accounts and clicks "Next"  
**And** the agency code "ATR4X2" is pre-filled and validated on Step 2  
**And** the user clicks "Next"  
**And** the user selects "Read & Write" on Step 3 and clicks "Next"  
**And** the user adds "teammate@brand.com" on Step 4 and clicks "Finish"  
**Then** "POST /api/flipkart/signup" is called with all collected data  
**And** on success the JWT is stored  
**And** the user is redirected to "/home"

# — Wizard Navigation —

@functional

#### Scenario: Stepper shows correct active step at each stage

**When** the user is on Step 1  
**Then** the stepper shows Step 1 as active  
**When** the user advances to Step 2  
**Then** the stepper shows Step 2 as active  
**When** the user advances to Step 3  
**Then** Step 3 is active  
**When** the user advances to Step 4  
**Then** Step 4 is active

@functional

#### Scenario: User can click back on stepper to revisit completed steps

**Given** the user has reached Step 3  
**When** the user clicks the Step 1 indicator in the stepper  
**Then** Step 1 is shown with previous selections intact

@functional

#### Scenario: User cannot jump forward to an incomplete step via stepper

**Given** the user is on Step 2  
**When** the user attempts to click Step 4 in the stepper  
**Then** the wizard does not advance to Step 4  
**And** Step 2 remains active

@functional

#### Scenario: Wizard state is preserved across all back-and-forth navigation

**Given** the user completes all 4 steps with specific values  
**When** the user navigates backwards through all steps and forwards again  
**Then** all previously entered values are still present at each step

# — API Submit —————

@functional

**Scenario: Loading state during final API submission**

**When** the user clicks "Finish" on Step 4  
**Then** a loading spinner or skeleton is displayed  
**And** the "Finish" button is disabled (no double submit)  
Until the API responds

@negative

**Scenario: Server error (500) on final submission**

**When** "POST /api/flipkart/signup" returns HTTP 500  
**Then** an error banner is shown on Step 4  
**And** the user remains on Step 4  
**And** the "Finish" button is re-enabled for retry

@negative

**Scenario: Duplicate account error (409) on final submission**

**When** "POST /api/flipkart/signup" returns HTTP 409 "Account already exists"  
**Then** the user is redirected to "/flipkart/login"  
**And** a message "An account with this email already exists. Please log in." is shown

@negative

**Scenario: Network timeout on final submission**

**When** "POST /api/flipkart/signup" times out  
**Then** a timeout error is shown  
**And** the "Finish" button is re-enabled  
**And** all wizard data is preserved for retry

# — UX / Accessibility —————

@ux @accessibility

**Scenario: Progress bar reflects current step**

As the user advances through steps  
**Then** the progress bar or stepper advances accordingly  
**And** the current step is visually distinct from completed and future steps

@accessibility

**Scenario: All form inputs have accessible labels**

**Given** the user is on any step of the signup wizard  
**Then** all input fields have associated label elements or aria-label attributes  
**And** the form is navigable via keyboard (Tab key)

@accessibility

**Scenario: Error messages are announced to screen readers**

**When** a validation error appears  
**Then** the error message has an appropriate "role=alert" or "aria-live" attribute

@ux

**Scenario: Browser back button behavior during signup**

**Given** the user is on Step 3 of the wizard

**When** the user presses the browser's back button

**Then** the user is either taken to Step 2 (if history is managed)

Or a confirmation dialog warns that progress may be lost

## 7. BE — Token Extraction

**Feature: Backend - extractEmailFromFlipkartToken()**

# — *Functional* —

**@smoke @functional @unit**

**Scenario: Standard email extraction from valid token**

**Given** a refresh token with sub claim "f:some-uuid:user@example.com;ads"  
**When** "extractEmailFromFlipkartToken()" is called  
**Then** it returns "user@example.com"

**@functional @unit**

**Scenario: Email with plus alias is extracted correctly**

**Given** a token with sub "f:uuid:sendto+interactiveavenue@email-l.ink;ads"  
**When** the function is called  
**Then** it returns "sendto+interactiveavenue@email-l.ink"

**@functional @unit**

**Scenario: Email with subdomain is extracted correctly**

**Given** a token with sub "f:uuid:admin@mail.flipkart.com;ads"  
**When** the function is called  
**Then** it returns "admin@mail.flipkart.com"

**@functional @unit**

**Scenario: Sub claim without ";ads" suffix is handled**

**Given** a token with sub "f:uuid:user@example.com"  
**When** the function is called  
**Then** it returns "user@example.com" without error

# — *Negative* —

**@negative @unit**

**Scenario: Token with missing "sub" claim throws error**

**Given** a JWT payload with no "sub" field: "{}"  
**When** the function is called  
**Then** an error is thrown with a descriptive message

**@negative @unit**

**Scenario: Malformed token (not a valid JWT) throws error**

**Given** input "not-a-jwt-string"  
**When** the function is called  
**Then** an error is thrown (cannot split/decode)

**@negative @unit**

**Scenario: Empty string token throws error**

**Given** input ""  
**When** the function is called

**Then** an error is thrown

@negative @unit

**Scenario: Token with only one JWT part throws error**

**Given** input "onlyonepart" (no dots)

**When** the function is called

**Then** an error is thrown

# — Boundary —————

@boundary @unit

**Scenario: Sub with UUID containing hyphens is parsed correctly**

**Given** sub "f:550e8400-e29b-41d4-a716-446655440000:user@example.com;ads"

**When** the function is called

**Then** it returns "user@example.com"

@boundary @unit

**Scenario: Email containing a colon character (edge parsing)**

**Given** sub "f:uuid:user:extra@example.com;ads"

**When** the function is called

**Then** it returns "user:extra@example.com" using slice(2).join(':')

# — Edge Cases —————

@edge @unit

**Scenario: Base64 padding issues in JWT payload**

**Given** a JWT payload that requires base64 padding ("=")

**When** the function decodes it

**Then** it decodes correctly without throwing an error

@edge @unit

**Scenario: JWT payload is URL-safe base64 encoded**

**Given** a JWT payload encoded with URL-safe base64 (using "-" and "\_")

**When** the function decodes it

**Then** it decodes correctly

@edge @unit

**Scenario: Very long email address in sub claim**

**Given** sub with a 254-character email address

**When** the function is called

**Then** the full email is returned correctly

## 8. BE — `GET /api/agency/validate-code/:code`

**Feature: Backend API - GET /api/agency/validate-code/:code**

# — Functional —

@smoke @functional @api

**Scenario: Valid agency code returns agency details**

**Given** the code "ATR4X2" exists in the database for an agency company  
**When** "GET /api/agency/validate-code/ATR4X2" is called  
**Then** the response status is 200  
**And** the response body contains "{ valid: true, agency\_name: '...', agency\_pk: '...' }"

@functional @api

**Scenario: Invalid agency code returns valid false**

**Given** the code "XXXXXX" does not exist in the database  
**When** "GET /api/agency/validate-code/XXXXXX" is called  
**Then** the response status is 200  
**And** the response body is "{ valid: false }"

@functional @api

**Scenario: Lowercase code is normalized to uppercase before lookup**

**Given** the code "ATR4X2" exists in the database  
**When** "GET /api/agency/validate-code/atrx2" is called  
**Then** it is treated the same as "ATR4X2"  
**And** the response is "{ valid: true, ... }"

@functional @api

**Scenario: Code belonging to a non-agency company returns valid false**

**Given** a code exists in "companies.agency\_code" but that company's type is "client"  
**When** that code is validated  
**Then** the response is "{ valid: false }"

# — Negative —

@negative @api

**Scenario: Code shorter than 6 characters returns valid false**

**When** "GET /api/agency/validate-code/ATR" is called (3 chars)  
**Then** the response status is HTTP 200  
**And** the response body is "{ valid: false }"  
**And** no 400 is thrown (regex /^[A-Z0-9]{6}\$/ fails, returns false)

@negative @api

**Scenario: Code longer than 6 characters returns valid false**

**When** "GET /api/agency/validate-code/ATR4X2ZZ" is called (8 chars)  
**Then** the response status is HTTP 200  
**And** the response body is "{ valid: false }"  
**And** no 400 is thrown (regex fails, returns false)

@negative @api

**Scenario: Code with special characters returns valid false**

**When** "GET /api/agency/validate-code/ATR-4!" is called  
**Then** the response status is HTTP 200  
**And** the response body is "{ valid: false }"  
**And** the regex /^[A-Z0-9]{6}\$/ fails after trim+uppercase normalization

@negative @api

**Scenario: Empty code path parameter returns error**

**When** "GET /api/agency/validate-code/" is called (empty code)  
**Then** the response is HTTP 400 or 404

# — Boundary —————

@boundary @api

**Scenario Outline: Code length boundary testing**

**When** "GET /api/agency/validate-code/<code>" is called  
**Then** the response status is "<status>"

**Examples:**

| code    | status | body                                  |
|---------|--------|---------------------------------------|
| AAAAA   | 200    | { valid: false }                      |
| AAAAAA  | 200    | { valid: true/false depending on DB } |
| AAAAAAA | 200    | { valid: false }                      |

# — Security —————

@security @api

**Scenario: SQL injection attempt in code parameter**

**When** "GET /api/agency/validate-code/'; DROP TABLE companies;--" is called  
**Then** the response status is HTTP 200  
**And** the response body is "{ valid: false }" (regex /^[A-Z0-9]{6}\$/ fails after normalization)  
**And** no database modification occurs

@security @api

**Scenario: Endpoint is public – no auth required**

**Given** no Authorization header is provided  
**When** the validate-code endpoint is called  
**Then** the response is not HTTP 401 or 403

@security @api

**Scenario: Rate limiting prevents brute-force enumeration of codes**

**When** the same IP makes more than 100 requests in 1 minute  
**Then** the API returns HTTP 429 Too Many Requests  
**And** further requests are throttled

# — Data Integrity —————

@data-integrity @api



**Scenario: Agency code lookup is case-insensitive but stored as uppercase**

**Given** the code "ATR4X2" is stored in uppercase in the database

**When** the API is called with "atr4x2"

**Then** it matches the uppercase stored value and returns valid true

## 9. BE — `POST /api/agency/generate-code`

**Feature: Backend API - POST /api/agency/generate-code**

### Background:

**Given** the caller is authenticated as an admin user

# — *Functional* —————

@smoke @functional @api

### Scenario: Generate code for a valid agency company

**Given** a company with "company\_type = 'agency'" and no existing agency\_code

**When** "POST /api/agency/generate-code" is called with that company's pk

**Then** the response contains a 6-character uppercase alphanumeric code

**And** the code is saved to "companies.agency\_code" for that company

@functional @api

### Scenario: Generated code is exactly 6 characters long

**When** a code is generated

**Then** the returned code has exactly 6 characters

@functional @api

### Scenario: Generated code is uppercase alphanumeric only

**When** a code is generated

**Then** it matches the pattern "[A-Z0-9]{6}"

**And** contains no lowercase letters or special characters

@functional @api

### Scenario: Agency code is generated once during initial setup and never overwritten

**Given** an agency is being onboarded for the first time and has no agency\_code

**When** "generateAndSaveAgencyCode" is called during initial agency setup

**Then** a unique 6-char code is generated and saved to "companies.agency\_code"

**And** this endpoint is not exposed as a re-generate endpoint

**And** calling it a second time for an agency that already has a code is not a valid use case

# — *Negative* —————

@negative @api

### Scenario: Cannot generate code for a non-agency company

**Given** a company with "company\_type = 'client'"

**When** "POST /api/agency/generate-code" is called for that company

**Then** the response is HTTP 400

**And** the error message says "Only agency companies can have codes"

@negative @api

### Scenario: Unauthenticated request is rejected

**Given** no Authorization header is provided

**When** "POST /api/agency/generate-code" is called

**Then** the response is HTTP 401

@negative @api

**Scenario: Non-admin authenticated user is rejected**

**Given** a user with "user\_type = 'viewer'" is authenticated  
**When** "POST /api/agency/generate-code" is called  
**Then** the response is HTTP 403

@negative @api

**Scenario: Missing company\_pk in request body**

**When** "POST /api/agency/generate-code" is called with no body  
**Then** the response is HTTP 400 with a validation error

@negative @api

**Scenario: Non-existent company\_pk returns error**

**When** called with a company\_pk that does not exist in the database  
**Then** the response is HTTP 404 or HTTP 400

# — Boundary / Uniqueness —————

@boundary @data-integrity @api

**Scenario: Generated codes are unique across multiple agencies**

**Given** 100 agency companies exist with no agency codes  
**When** codes are generated for all 100  
**Then** all 100 codes are distinct (no duplicates)

@boundary @api

**Scenario: Code generation retries on collision**

**Given** the randomly generated code already exists in the database  
**When** code generation is attempted  
**Then** the system retries and generates a different unique code  
**And** does not save a duplicate

# — Edge Cases —————

@edge @api

**Scenario: Concurrent code generation for two agencies doesn't produce duplicates**

**Given** two parallel requests to generate codes for two different agencies  
**When** both requests complete simultaneously  
**Then** each agency receives a different unique code  
**And** the uniqueness constraint is not violated

## 10. BE — `POST /api/flipkart/signup`

**Feature: Backend API - POST /api/flipkart/signup**

### Background:

**Given** the "POST /api/flipkart/signup" endpoint is available

# — Happy Path —————

@smoke @functional @api

### Scenario: Full happy path — Read permission signup

**Given** a valid request body with:

| field             | value                           |
|-------------------|---------------------------------|
| auth_code         | VALID_AUTH_CODE                 |
| selected_accounts | [{ ad_account_id: "acc1" ... }] |
| agency_code       | ATR4X2                          |
| permission_level  | read                            |

**When** the endpoint is called

**Then** the response is HTTP 200

**And** "success: true" is in the response

**And** a valid JWT is returned in "token"

**And** a company with "company\_type = 'client'" is created

**And** a user with "user\_type = 'admin'" is created

**And** connected\_accounts are created for each selected account

**And** a workspace is created

**And** an "agency\_clients" record is created with "permission\_level = 'read'"

**And** 2 service tokens are created per connected account (1 agency token + 1 client token)

**And** all service tokens are for the national platform only (grocery and minute not yet active)

**And** total service tokens = 2 × number of selected accounts

@boundary @api

### Scenario: Service token count scales with number of selected accounts

**Given** a signup request with 1 selected account

**When** signup completes successfully

**Then** 2 service tokens are created (1 agency + 1 client)

@boundary @api

### Scenario: Service token count with multiple accounts

**Given** a signup request with 5 selected accounts

**When** signup completes successfully

**Then** 10 service tokens are created (2 per account)

@smoke @functional @api

### Scenario: Full happy path — Write permission signup

**Given** a valid request body with "permission\_level: 'write'"

**When** the endpoint is called

**Then** "agency\_clients.permission\_level = 'write'" is saved

@functional @api

### Scenario: Signup with team invitations

**Given** the request includes "invited\_emails: ['a@b.com', 'c@d.com']"

**When** the endpoint is called successfully  
**Then** invitation emails are sent to "a@b.com" and "c@d.com"

@functional @api

**Scenario: Signup without team invitations (omitted field)**

**Given** the request does not include "invited\_emails"  
**When** the endpoint is called  
**Then** signup completes successfully  
**And** no invitation emails are sent

@functional @api

**Scenario: JWT returned contains correct Hasura claims**

**When** signup completes successfully  
**Then** the decoded JWT contains:

| claim                 | value            |
|-----------------------|------------------|
| x-hasura-company-id   | <new-company-pk> |
| x-hasura-user-id      | <new-user-pk>    |
| x-hasura-default-role | client_admin     |

@functional @api

**Scenario: Agency notification email is sent on successful signup**

**Given** a valid signup request with agency\_code "ATR4X2"  
**When** the signup completes  
**Then** an email is sent to all admin users of the "ATR4X2" agency  
**And** the email contains the brand name, brand email, and permission level

# — Negative —————

@negative @api

**Scenario: Invalid or expired auth\_code**

**Given** the request has an expired "auth\_code"  
**When** the endpoint is called  
**Then** the response is HTTP 400  
**And** the response body is "{ success: false, error: 'Failed to exchange auth code' }"

@negative @api

**Scenario: Invalid agency\_code**

**Given** the request has "agency\_code: 'XXXXXX'" which does not exist  
**When** the endpoint is called  
**Then** the response is HTTP 400  
**And** the response body is "{ success: false, error: 'Invalid agency code' }"

@negative @api

**Scenario: Empty selected\_accounts array**

**Given** the request has "selected\_accounts: []"  
**When** the endpoint is called  
**Then** the response is HTTP 400  
**And** the response body is "{ success: false, error: 'At least one account required' }"

@negative @api

**Scenario: Missing auth\_code field**

**Given** the request body has no "auth\_code"  
**When** the endpoint is called  
**Then** the response is HTTP 400 with a validation error

@negative @api

**Scenario: Missing agency\_code field**

**Given** the request body has no "agency\_code"  
**When** the endpoint is called  
**Then** the response is HTTP 400 with a validation error

@negative @api

**Scenario: Missing selected\_accounts field**

**Given** the request body has no "selected\_accounts"  
**When** the endpoint is called  
**Then** the response is HTTP 400 with a validation error

@negative @api

**Scenario: Missing permission\_level field**

**Given** the request body has no "permission\_level"  
**When** the endpoint is called  
**Then** the response is HTTP 400 with a validation error

@negative @api

**Scenario: Invalid permission\_level value**

**Given** the request has "permission\_level: 'admin'"  
**When** the endpoint is called  
**Then** the response is HTTP 400  
**And** the error mentions invalid permission\_level value

@negative @api

**Scenario: Duplicate user — email already exists**

**Given** the email extracted from the auth token already exists in "users"  
**When** the endpoint is called  
**Then** the response is HTTP 409  
**And** the response body is "{ success: false, error: 'Account already exists' }"  
**And** no new company, user, or workspace is created

# — Boundary —————

@boundary @api

**Scenario: Signup with exactly 1 selected account**

**Given** "selected\_accounts" has exactly 1 account  
**When** signup completes  
**Then** 1 connected\_account is created

@boundary @api

### Scenario: Signup with maximum number of selected accounts

**Given** "selected\_accounts" has 50 accounts  
**When** signup completes  
**Then** all 50 connected\_accounts are created  
**And** the workspace includes all 50

@boundary @api

### Scenario Outline: permission\_level only accepts 'read' or 'write'

**Given** "permission\_level" is set to "<value>"  
**Then** the response is "<expected>"

#### Examples:

|        |                      |  |
|--------|----------------------|--|
| value  | expected             |  |
| read   | 200 success          |  |
| write  | 200 success          |  |
| Read   | 400 validation error |  |
| WRITE  | 400 validation error |  |
| admin  | 400 validation error |  |
| viewer | 400 validation error |  |
| ""     | 400 validation error |  |

# — Data Integrity —————

@data-integrity @api

### Scenario: All signup steps are atomic – failure triggers soft-delete rollback

**Given** signup processing fails during workspace creation (step 7 of 11)  
**When** the endpoint's inner try-catch catches the error  
**Then** softDeleteUserMutation is called to soft-delete any created user record  
**And** softDeleteCompanyMutation is called to soft-delete any created company record  
**And** the error is re-thrown returning HTTP 500  
**And** the following tables contain no committed partial data:

|                    |                                   |  |
|--------------------|-----------------------------------|--|
| table              | expectation                       |  |
| companies          | soft-deleted or no record created |  |
| users              | soft-deleted or no record created |  |
| connected_accounts | no new records created            |  |
| workspace          | no new record created             |  |
| agency_clients     | no new record created             |  |
| service_tokens     | no new records created            |  |

@data-integrity @api

### Scenario: Duplicate ad account connected to another company – warning logged, proceed

**Given** one of the "selected\_accounts" has an "ad\_account\_id" already in another company  
**When** signup is called  
**Then** signup proceeds and completes successfully  
**And** a warning is logged server-side  
**And** the upsert is performed

@data-integrity @api

### Scenario: Email is extracted from token sub claim, not from request body

**Given** a valid auth\_code  
**When** the endpoint decodes the refresh token  
**Then** the user's email is taken from the "sub" claim of the refresh token  
**And** no email field is accepted from the request body

# — Edge Cases —

@edge @api

**Scenario: invited\_emails contains the brand owner's own email**

**Given** "invited\_emails" includes the same email as the signing-up brand user  
**When** the endpoint is called  
**Then** the signup completes successfully  
**And** the invitation email is sent to the brand owner's email address  
**But** no duplicate user record is created (owner is already created in step 4)  
**And** the existing user check at login handles the recipient gracefully

@edge @api

**Scenario: invited\_emails contains duplicate emails**

**Given** "invited\_emails: ['a@b.com', 'a@b.com']"  
**When** the endpoint is called  
**Then** only one invitation is sent to "a@b.com"  
**And** deduplication occurs server-side

@edge @api

**Scenario: Auth code is exchanged exactly once – not reused**

**Given** a valid "auth\_code" is submitted  
**When** the endpoint successfully processes the signup  
**And** the same "auth\_code" is submitted again  
**Then** the second request fails with HTTP 400 (code already used)

@edge @api

**Scenario: Flipkart token exchange succeeds but sub claim is malformed**

**Given** the exchange returns a token with a malformed sub claim  
**When** "extractEmailFromFlipkartToken()" throws an error  
**Then** the signup fails with HTTP 400  
**And** no partial records are created

@edge @api

**Scenario: User whose previous signup failed midway attempts to sign up again**

**Given** a brand's previous signup attempt failed mid-way  
**And** the failure triggered softDeleteUserMutation and softDeleteCompanyMutation  
**And** the orphaned records were soft-deleted  
**When** the same brand attempts signup again with the same email from the OAuth token  
**Then** the soft-deleted records do not block the new signup  
**And** a fresh set of company, user, workspace, and agency\_clients records is created  
**And** the signup completes successfully returning a valid JWT



## 11. BE — `POST /api/auth/flipkart-sso` (Modified)

**Feature: Backend API - POST /api/auth/flipkart-sso (Modified Behavior)**

# — Functional —————

@smoke @functional @api

**Scenario: Existing onboarded user receives JWT**

**Given** a valid auth\_code for a user whose ad\_account\_ids are in the database  
**When** "POST /api/auth/flipkart-sso" is called  
**Then** the response is HTTP 200  
**And** the response contains "{ onboarded: true, token: '<jwt>' }"  
**And** the JWT is valid and contains correct Hasura claims

@smoke @functional @api

**Scenario: New user — returns onboarded false, does NOT create records**

**Given** a valid auth\_code for a user not found by ad\_account\_ids  
**When** "POST /api/auth/flipkart-sso" is called  
**Then** the response is HTTP 200  
**And** the response contains "{ onboarded: false }"  
**And** no company, user, or workspace record is created in the database

@functional @api

**Scenario: Existing user's tokens are refreshed on login**

**Given** an existing user with expired service\_tokens  
**When** "POST /api/auth/flipkart-sso" is called  
**Then** the user's service\_tokens are updated with fresh Flipkart tokens  
**And** a new JWT is returned

# — Negative —————

@negative @api

**Scenario: Invalid auth\_code returns error**

**Given** an expired or invalid "auth\_code"  
**When** "POST /api/auth/flipkart-sso" is called  
**Then** the response is HTTP 400  
**And** an error message is returned

@negative @api

**Scenario: Missing auth\_code in request body**

**Given** the request body has no "auth\_code"  
**When** the endpoint is called  
**Then** the response is HTTP 400 with a validation error

# — State —————

@state @api

**Scenario: Old behavior (auto-create company) no longer occurs**

**Given** a new user not found in the database

**When** "POST /api/auth/flipkart-sso" is called  
**Then** no placeholder company is auto-created  
**And** the response is "{ onboarded: false }"  
**And** this is the only code path for new users

# — Security —————

@security @api

**Scenario: Cannot replay an already-used auth\_code**

**Given** an auth\_code was already used in a prior SSO call  
**When** the same code is submitted again  
**Then** the response is HTTP 400 (code expired or already consumed)

## 12. DB — Database Migrations

### Feature: Database Migrations

# — *companies.agency\_code* —————

@smoke @db

#### Scenario: *agency\_code* column is added to *companies* table

**Given** the migration "add\_agency\_code\_to\_companies" is applied  
**Then** "companies.agency\_code" column exists  
**And** it is of type VARCHAR(6)  
**And** it is nullable  
**And** it has a UNIQUE constraint

@db

#### Scenario: *agency\_code* UNIQUE constraint is enforced

**Given** two agency companies exist  
**When** the same 6-char code is inserted for both  
**Then** the second insert fails with a unique constraint violation

@db

#### Scenario: *agency\_code* allows NULL for non-agency companies

**Given** a company with "company\_type = 'client'"  
**Then** "agency\_code" can be NULL for that company

@db

#### Scenario: *agency\_code* only populated for agency companies

**Given** the backfill migration runs  
**Then** all companies with "company\_type = 'agency'" receive a non-null *agency\_code*  
**And** companies with "company\_type != 'agency'" have NULL *agency\_code*

# — *agency\_clients.permission\_level* —————

@smoke @db

#### Scenario: *permission\_level* column is added to *agency\_clients* table

**Given** the migration "add\_permission\_level\_to\_agency\_clients" is applied  
**Then** "agency\_clients.permission\_level" column exists  
**And** it is of type VARCHAR(10)  
**And** it is NOT NULL  
**And** it has DEFAULT value 'read'  
**And** it has a CHECK constraint allowing only 'read' or 'write'

@db

#### Scenario Outline: *permission\_level* CHECK constraint validation

**When** a record is inserted with "permission\_level = '<value>'"  
**Then** the result is "<outcome>"

#### Examples:

| value | outcome |
|-------|---------|
|       |         |

|        |                                  |  |
|--------|----------------------------------|--|
| read   | Insert succeeds                  |  |
| write  | Insert succeeds                  |  |
| admin  | CHECK constraint violation error |  |
| viewer | CHECK constraint violation error |  |
| READ   | CHECK constraint violation error |  |
| WRITE  | CHECK constraint violation error |  |
| ""     | CHECK constraint violation error |  |
| NULL   | NOT NULL constraint violation    |  |

@db

#### Scenario: Existing agency\_clients records default to 'read' after migration

**Given** existing "agency\_clients" records before migration  
**When** the migration adds the permission\_level column  
**Then** all existing records have "permission\_level = 'read'"

# — Backfill —————

@db

#### Scenario: Backfill generates unique 6-char codes for all agency companies

**Given** 50 agency companies exist without agency\_code  
**When** the backfill script runs  
**Then** all 50 companies have a non-null agency\_code  
**And** all 50 codes are unique  
**And** all codes match the pattern "[A-Z0-9]{6}"

@db

#### Scenario: Backfill does not overwrite existing agency codes

**Given** one agency already has "agency\_code = 'ATR4X2'"  
**When** the backfill script runs  
**Then** that agency's code remains "ATR4X2"

@db

#### Scenario: Backfill script is fully idempotent – safe to re-run after partial failure

**Given** the backfill script ran and generated codes for 30 out of 50 agencies before failing  
**When** the backfill script is re-run  
**Then** it queries only agencies where "agency\_code: { \_is\_null: true }"  
**And** the 30 agencies that already have codes are skipped  
**And** the remaining 20 agencies receive new unique codes  
**And** no existing code is overwritten

# — Rollback —————

@db

#### Scenario: Rollback of agency\_code migration removes the column

**Given** the migration "add\_agency\_code\_to\_companies" has been applied  
**When** the down migration is run  
**Then** "companies.agency\_code" column no longer exists

@db

#### Scenario: Rollback of permission\_level migration removes the column

**Given** the migration "add\_permission\_level\_to\_agency\_clients" has been applied  
**When** the down migration is run  
**Then** "agency\_clients.permission\_level" column no longer exists

## 13. Email Notifications

### Feature: Email Notifications

#### Background:

**Given** a brand successfully completes signup with agency\_code "ATR4X2"  
**And** the agency "MediaCom Agency" has 3 admin users

# — Functional —————

@smoke @functional @email

#### Scenario: Agency admins receive "new brand joined" email

**Then** all 3 admin users of "MediaCom Agency" receive the email  
**And** the email uses the "newBrandJoined" template  
**And** the email subject references the new brand

@functional @email

#### Scenario: "New brand joined" email contains required information

**When** an admin opens the notification email  
**Then** it contains the brand's company name  
**And** it contains the brand's email address  
**And** it contains the permission level granted ("read" or "write")

@functional @email

#### Scenario: Team invitation email is sent to invited users

**Given** the signup included "invited\_emails: ['newuser@brand.com']"  
**Then** "newuser@brand.com" receives an invitation email  
**And** the email contains a signup/join link

# — Negative —————

@negative @email

#### Scenario: No agency notification when signup has no agency\_code

**Given** a signup completes without an agency\_code (if allowed)  
**Then** no "new brand joined" email is sent

@negative @email

#### Scenario: Email failure does not break signup flow

**Given** the email service is temporarily unavailable  
**When** a brand completes signup  
**Then** the signup still completes and returns a JWT  
**And** the email failure is logged server-side  
**And** the user is not shown an error due to email failure

@negative @email

#### Scenario: Agency with no admin users — no email sent but no crash

**Given** the agency has 0 users with "user\_type = 'admin'"  
**When** a brand signs up with that agency's code  
**Then** signup completes successfully

**And** no email sending is attempted (or 0 emails are sent)  
**And** no error occurs

# — *Edge Cases* —————

@edge @email

**Scenario: Agency has 100 admin users — all receive the notification**

**Given** the agency has 100 admin users  
**When** a brand signs up  
**Then** all 100 admins receive the "new brand joined" email

@edge @email

**Scenario: Invited email address belongs to an already-registered user**

**Given** "invited\_emails" contains an email already registered in the system  
**When** signup completes  
**Then** the invitation email is sent regardless (fire-and-forget notification)  
**And** no error is thrown and no duplicate user record is created  
**And** the existing user can simply log in using the link in the invitation email

@edge @email

**Scenario: Invitation email links are not reusable after first use**

**Given** an invitation email was sent to "newuser@brand.com"  
**When** the invited user uses the link to sign up  
**And** someone else tries to use the same link  
**Then** the second use of the link fails with an "invitation expired or already used" message

## 14. Security

### Feature: Security - Cross-Cutting Concerns

# — Authentication & Authorization —

@security

#### Scenario: Agency validate-code endpoint is public – no auth required

**Given** no Authorization header is provided  
**When** "GET /api/agency/validate-code/ATR4X2" is called  
**Then** the response is not 401 or 403

@security

#### Scenario: Generate-code endpoint requires admin authentication

**Given** no Authorization header is provided  
**When** "POST /api/agency/generate-code" is called  
**Then** the response is HTTP 401

@security

#### Scenario: Brand user cannot access another brand's workspace

**Given** Brand A signs up and receives a JWT  
**When** Brand A's JWT is used to query Brand B's workspace data  
**Then** the response is HTTP 403

@security

#### Scenario: Agency user with Read permission cannot mutate data

**Given** an agency user has "permission\_level = 'read'" on a brand's workspace  
**When** the agency user attempts to create/edit a campaign  
**Then** the action is blocked with HTTP 403

@security

#### Scenario: Agency user with Write permission cannot perform destructive actions

**Given** an agency user has "permission\_level = 'write'"  
**When** the agency user attempts to disconnect a connected account  
**Then** the action is blocked with HTTP 403  
**When** the agency user attempts to delete the workspace  
**Then** the action is blocked with HTTP 403

# — Input Validation & Injection —

@security

#### Scenario: SQL injection in agency code field is rejected

**When** "GET /api/agency/validate-code/'; DROP TABLE companies;--" is called  
**Then** the database is not affected  
**And** the response is 400 or "{ valid: false }"

@security

#### Scenario: XSS attempt in invite email field is sanitized



**When** the user types "<script>alert('xss')</script>" in the invite email input  
**Then** the input is rejected or sanitized  
**And** no script is executed

@security

**Scenario: XSS attempt in agency code field is handled**

**When** "<img src=x onerror=alert(1)>" is entered in the agency code input  
**Then** the input is sanitized or rejected  
**And** no script is executed

@security

**Scenario: JWT token cannot be forged**

**Given** an attacker crafts a JWT with "x-hasura-default-role: agency\_admin"  
**When** the crafted JWT is used to call a protected API  
**Then** the server rejects it with HTTP 401  
**And** the server verifies the signature correctly

@security

**Scenario: Sensitive data not included in API error messages**

**Given** an unexpected server error occurs  
**When** the error response is returned  
**Then** it does not expose stack traces, database schema, or internal paths

# — Token Security —————

@security

**Scenario: Flipkart OAuth auth code cannot be replayed**

**Given** a valid auth\_code is used once for SSO login  
**When** the same auth\_code is submitted again  
**Then** the server returns an error (code already consumed or expired)

@security

**Scenario: JWT has an appropriate expiry time**

**Given** a JWT is returned from signup or SSO login  
**When** the JWT payload is decoded  
**Then** the "exp" claim is set to a reasonable future time  
**And** the JWT is not set to never expire

@security

**Scenario: CSRF protection on state-mutating endpoints**

**Given** a cross-site request is made to "POST /api/flipkart/signup" without proper origin  
**Then** the server validates the origin/CSRF token  
**And** unauthorized cross-site requests are rejected

# — Brute Force Protection —————

@security

**Scenario: Agency code validation endpoint is rate-limited**

**When** the same IP makes 200 requests to validate different codes within 1 minute  
**Then** requests are throttled with HTTP 429  
**And** an attacker cannot enumerate all valid agency codes

@security

**Scenario: SSO endpoint is rate-limited**

**When** the same IP makes excessive calls to "POST /api/auth/flipkart-sso"  
**Then** the endpoint rate-limits or blocks excessive requests

# — Permission Escalation —————

@security

**Scenario: Agency Read user attempts write operation via direct crafted API call**

**Given** an agency user has "permission\_level = 'read'" on a brand workspace  
**When** they craft a direct API request to create a campaign (bypassing the UI)  
**Then** the response is HTTP 403  
**And** no campaign record is created in the database  
**And** the agency\_clients.permission\_level remains unchanged

@security

**Scenario: Agency Read user attempts to modify their own permission\_level via API**

**Given** an agency user has "permission\_level = 'read'"  
**When** they send a direct API request to update their own permission to "write"  
**Then** the response is HTTP 403  
**And** the permission\_level in agency\_clients remains "read"

@security

**Scenario: Agency Write user attempts to perform a destructive action via direct API call**

**Given** an agency user has "permission\_level = 'write'"  
**When** they craft a direct API request to delete the workspace  
**Then** the response is HTTP 403  
**And** the workspace is not deleted  
**When** they craft a direct API request to disconnect a connected account  
**Then** the response is HTTP 403  
**And** the connected account remains intact

@security

**Scenario: Brand user attempts to access another brand's workspace via direct API**

**Given** Brand A is authenticated with their JWT  
**When** Brand A crafts an API call using Brand B's workspace ID  
**Then** the response is HTTP 403  
**And** no data from Brand B's workspace is returned

@security

**Scenario: Agency user attempts to access a brand workspace they are not linked to**

**Given** an agency user is linked to Brand A's workspace  
**When** they craft an API call using Brand B's workspace ID  
**Then** the response is HTTP 403  
**And** no data from Brand B's workspace is returned



## 15. Concurrency & Race Conditions

### Feature: Concurrency and Race Conditions

#### @concurrency

##### Scenario: Two users sign up with the same email simultaneously

**Given** two separate HTTP requests are made to "POST /api/flipkart/signup"  
**And** both requests decode the same user email from their tokens  
**When** both requests are processed concurrently  
**Then** only one user record is created in the database  
**And** the second request receives HTTP 409 "Account already exists"  
**And** no duplicate user records exist

#### @concurrency

##### Scenario: Two agencies generate codes simultaneously – no duplicates

**Given** two requests to "POST /api/agency/generate-code" arrive at the same time  
**When** both requests complete  
**Then** each agency receives a different unique code  
**And** the database UNIQUE constraint is never violated

#### @concurrency

##### Scenario: User completes signup in two browser tabs simultaneously

**Given** the same user has two browser tabs open on "/flipkart/signup"  
**When** both tabs submit the final form at nearly the same time  
**Then** only one signup completes successfully  
**And** the second returns HTTP 409  
**And** no duplicate records are created

#### @concurrency

##### Scenario: Agency code is validated and then deleted before signup completes

**Given** a user validates agency code "ATR4X2" on Step 2  
**And** the agency code is deleted from the database before the user submits  
**When** "POST /api/flipkart/signup" is called  
**Then** the signup fails with HTTP 400 "Invalid agency code"  
**And** no partial records are created

#### @concurrency

##### Scenario: Multiple brands join the same agency simultaneously

**Given** 5 brands submit signup requests with the same agency code simultaneously  
**When** all 5 requests complete  
**Then** 5 separate "agency\_clients" records are created  
**And** all 5 agency admin notification emails are sent  
**And** no data corruption occurs

#### @concurrency

##### Scenario: User refreshes the page mid-signup – no duplicate submissions

**Given** the user is on Step 4 and clicks "Finish"  
**When** the page is refreshed immediately after  
**Then** the API is called only once  
**And** no duplicate signup is created



## 16. Integration / End-to-End

### Feature: Integration and End-to-End Flows

# — Full Brand Onboarding —

@smoke @e2e @integration

#### Scenario: Complete new brand onboarding journey

**Given** a new Flipkart seller visits "/flipkart/login"  
**When** they click "Login with Flipkart" and complete OAuth  
**And** the SSO API returns "{ onboarded: false }"  
**And** they are redirected to "/flipkart/signup?code=AUTH\_CODE"  
**And** they select 2 ad accounts on Step 1  
**And** they enter valid agency code "ATR4X2" on Step 2  
**And** they select "Read & Write" permission on Step 3  
**And** they add 1 teammate email and click "Finish" on Step 4  
**Then** the brand can access "/home"  
**And** the agency admin can see the new brand in their client list  
**And** the agency admin received a "new brand joined" email  
**And** the invited teammate received an invitation email

@e2e @integration

#### Scenario: Returning brand logs in directly to /home

**Given** a brand has previously completed onboarding  
**When** they visit "/flipkart/login" and complete OAuth  
**And** the SSO API returns "{ onboarded: true, token: '<jwt>' }"  
**Then** they are immediately redirected to "/home"  
**And** no signup steps are shown

# — Agency Access Post-Onboarding —

@e2e @integration

#### Scenario: Agency views brand workspace with Read-only permission

**Given** a brand signed up with "permission\_level: 'read'"  
**When** an agency user accesses that brand's workspace  
**Then** the agency user can view reports, campaigns, and dashboards  
**And** all edit/create/delete buttons are disabled or hidden  
**And** attempting any mutation via API returns HTTP 403

@e2e @integration

#### Scenario: Agency manages brand workspace with Write permission

**Given** a brand signed up with "permission\_level: 'write'"  
**When** an agency user accesses that brand's workspace  
**Then** the agency user can create and edit campaigns  
**And** can manage budgets, tags, and creatives  
**And** can invite users to the workspace  
**But** cannot disconnect connected accounts  
**And** cannot delete the workspace

# — Permission Access Matrix Validation —

@e2e @integration

### Scenario Outline: Read permission blocks all write operations

**Given** the agency has "permission\_level: 'read'" on a brand workspace  
**When** the agency user attempts to "<action>"  
**Then** the action is "<result>"

#### Examples:

| action                    | result  |
|---------------------------|---------|
| Edit budget allocations   | Blocked |
| Create a campaign         | Blocked |
| Create a tag              | Blocked |
| Edit workspace settings   | Blocked |
| Invite user to workspace  | Blocked |
| Edit budget constraints   | Blocked |
| Create Madison Plan       | Blocked |
| Disconnect accounts       | Blocked |
| Delete workspace          | Blocked |
| View reports              | Allowed |
| Run chatbot query         | Allowed |
| View campaign performance | Allowed |

@e2e @integration

### Scenario Outline: Write permission allows management actions but blocks destructive ones

**Given** the agency has "permission\_level: 'write'" on a brand workspace  
**When** the agency user attempts to "<action>"  
**Then** the action is "<result>"

#### Examples:

| action                   | result  |
|--------------------------|---------|
| Edit budget allocations  | Allowed |
| Create a campaign        | Allowed |
| Create a tag             | Allowed |
| Edit workspace settings  | Allowed |
| Invite user to workspace | Allowed |
| Create Madison Plan      | Allowed |
| Disconnect accounts      | Blocked |
| Delete workspace         | Blocked |

# — Invitation Link Flow —————

@e2e @integration

### Scenario: Brand arrives via invitation link with pre-filled agency code

**Given** an agency shares the URL "/flipkart/signup?agency\_code=ATR4X2"  
**When** a brand visits that URL  
**And** completes OAuth on the login/signup page  
**Then** on Step 2, the agency code "ATR4X2" is pre-filled  
**And** the agency name is automatically shown  
**And** the brand can proceed without manually entering the code

@e2e @integration

### Scenario: Team invitation completes successfully

**Given** an invited teammate received an invitation email  
**When** they click the invitation link  
**Then** they can complete their own account setup

**And** they gain access to the brand's workspace  
**And** no new company is created (they join the existing workspace)

# — *Data Consistency Across Systems* —

@e2e @integration @data-integrity

**Scenario: All entities created during signup are linked correctly**

**When** a brand successfully signs up  
**Then** the created "user" is linked to the created "company"  
**And** the "company" is linked to the "workspace"  
**And** the "connected\_accounts" are linked to the "workspace"  
**And** the "agency\_clients" record links the brand company to the agency company  
**And** the "service\_tokens" are linked to the brand company  
**And** all foreign key relationships are valid and non-null



## 17. Invited Teammate Journey (Gap — P0)

**Feature: Invited Teammate Authentication and Onboarding**

# — Magic Link / Non-SSO Authentication —

@smoke @functional @e2e

**Scenario: Invited teammate signs up via magic link (not Flipkart SSO)**

**Given** "teammate@brand.com" was invited during brand onboarding Step 4  
**And** they received an invitation email containing a magic link  
**When** they click the magic link  
**Then** they land on an appropriate signup or account setup page  
**And** they do NOT need to go through Flipkart OAuth  
**And** they do NOT need to select ad accounts or enter an agency code  
**And** their account is linked to the brand's existing workspace automatically  
**And** they gain access to the workspace without creating a new company

@functional @e2e

**Scenario: Invited teammate tries to sign up via Flipkart SSO instead of magic link**

**Given** "teammate@brand.com" received an invitation email with a magic link  
**When** they navigate to "/flipkart/login" and complete Flipkart OAuth instead  
**Then** the system recognises them as an invited user by their email  
**And** links them to the correct brand workspace  
**And** does not put them through the full 4-step onboarding wizard  
**And** does not create a new company for them

@negative

**Scenario: Invited teammate ignores magic link and attempts fresh standalone signup**

**Given** "teammate@brand.com" was invited but never clicked the link  
**When** they visit "/flipkart/signup" independently as a new user  
**Then** the system either links them to the existing workspace  
Or treats them as a new brand user going through full onboarding  
**And** the behaviour is explicitly defined – not ambiguous or silent

@negative

**Scenario: Invited teammate clicks magic link after it has expired**

**Given** "teammate@brand.com" received an invitation email  
**And** the invitation link has expired (TTL exceeded)  
**When** they click the expired magic link  
**Then** they see a clear error "This invitation link has expired"  
**And** they are shown instructions to request a new invitation

@negative

**Scenario: Magic link is used by a different person than the invitee**

**Given** an invitation link was sent to "teammate@brand.com"  
**When** someone with email "attacker@other.com" clicks the same link  
**Then** access is denied  
**And** only "teammate@brand.com" can use that invitation link

@edge

**Scenario: Invited teammate clicks magic link on a different device or browser**

**Given** "teammate@brand.com" received the invitation on mobile  
**When** they click the magic link on a desktop browser  
**Then** the invitation flow works correctly  
**And** session is established on the new device

@edge

**Scenario: Brand owner re-invites the same email after first invitation was ignored**

**Given** "teammate@brand.com" was invited but never signed up  
**When** the brand owner sends a new invitation to "teammate@brand.com"  
**Then** a fresh invitation link is generated  
**And** the old link is invalidated  
**And** the new link works correctly

@concurrency

**Scenario: Two people click the same magic link simultaneously**

**Given** one invitation link was sent to "teammate@brand.com"  
**When** the same link is clicked simultaneously from two different browsers  
**Then** only one session is created  
**And** the second click is handled gracefully (not a duplicate user)

# — Post-Join State —————

@functional

**Scenario: Invited teammate's role and permissions after joining**

**Given** "teammate@brand.com" successfully joins via invitation link  
**Then** they are created as a user linked to the brand's company  
**And** they have the correct default role within the workspace  
**And** they can access the workspace immediately after joining

@functional

**Scenario: Multiple teammates invited in one batch join independently**

**Given** 3 teammates were invited: "a@brand.com", "b@brand.com", "c@brand.com"  
**When** each clicks their own invitation link independently  
**Then** all 3 are linked to the same brand workspace  
**And** each has their own separate user account  
**And** no invitation link from one works for another

@data-integrity

**Scenario: Invited teammate joining does not create a new company**

**Given** "teammate@brand.com" accepts invitation and joins  
**Then** no new company record is created  
**And** no new workspace is created  
**And** they are added to the existing brand company and workspace only

## 18. Gaps — Added P0 Scenarios

### Feature: Wizard Step Enforcement — Server-Side Validation

@security @functional

**Scenario: User navigates directly to signup URL and tries to POST without completing wizard**

**Given** a user navigates directly to "/flipkart/signup" without going through Step 1  
**When** they craft a direct POST to "/api/flipkart/signup" with missing wizard fields  
**Then** the server returns HTTP 400 with validation errors  
**And** completion of all required fields is enforced server-side, not just client-side

@security

**Scenario: User bypasses Step 2 via direct URL and skips agency code**

**Given** a user completes Step 1 and tries to jump to Step 3 via direct URL manipulation  
**When** the wizard state is missing Step 2 data (agency\_code)  
**Then** the wizard does not allow jumping ahead  
**And** the POST to "/api/flipkart/signup" without agency\_code returns HTTP 400

### Feature: Signup Endpoint Idempotency

@edge @data-integrity

**Scenario: Client retries signup POST due to network timeout — no duplicate records**

**Given** the first POST to "/api/flipkart/signup" times out on the client  
**But** the server actually processed it successfully  
**When** the client retries the same request  
**Then** only one set of records is created  
**And** no duplicate company, user, or workspace is created  
**And** the second call returns a success response with the same JWT

@edge @data-integrity

**Scenario: Signup is retried after a soft-delete rollback — fresh records created**

**Given** a previous signup attempt failed and triggered soft-delete rollback  
**When** the same user retries signup with the same auth code (or new OAuth)  
**Then** new company, user, and workspace records are created cleanly  
**And** the soft-deleted records do not interfere

### Feature: Soft-Deleted Records Visibility

@data-integrity @security

**Scenario: Soft-deleted company is not visible in normal queries**

**Given** a company was soft-deleted after a failed signup  
**When** any API queries the companies table normally  
**Then** the soft-deleted company does not appear in results  
**And** it cannot be linked to new signups or agency clients

@data-integrity

**Scenario: Soft-deleted user cannot log in**

**Given** a user record was soft-deleted after a failed signup rollback  
**When** that email is used to attempt SSO login  
**Then** the system does not find the soft-deleted user  
**And** returns "{ onboarded: false }" – allowing a fresh signup

**Feature: Email Deduplication – Case Sensitivity**

@edge @negative

**Scenario: invited\_emails contains same address in different casing**

**Given** "invited\_emails: ['user@test.com', 'User@Test.COM', 'USER@TEST.COM']"  
**When** the signup API processes the list  
**Then** deduplication is case-insensitive  
**And** only one invitation email is sent to "user@test.com"

@edge @negative

**Scenario: Brand owner's email matches invited email in different casing**

**Given** the brand owner's email is "owner@brand.com"  
**And** "invited\_emails" contains "Owner@Brand.COM"  
**When** the signup API processes the list  
**Then** the case-insensitive match is detected  
**And** the invitation is sent but no duplicate user is created

**Feature: Service Token Correction – 2 Tokens Per Account (National Only)**

@smoke @functional @api

**Scenario: Signup with 1 account creates exactly 2 service tokens**

**Given** the user selects 1 national ad account during signup  
**When** signup completes successfully  
**Then** exactly 2 service tokens are created (1 agency token + 1 client token)  
**And** both tokens are linked to that 1 connected account

@boundary @api

**Scenario: Signup with N accounts creates exactly 2 x N service tokens**

**Given** the user selects 3 national ad accounts during signup  
**When** signup completes successfully  
**Then** exactly 6 service tokens are created (2 per account)  
**And** each pair of tokens is correctly linked to its respective account

@functional @api

**Scenario: Only national platform tokens are created during onboarding**

**Given** the user completes signup selecting national ad accounts  
**When** signup completes  
**Then** service tokens are created for national platform only  
**And** no grocery or minute platform tokens are created at this stage

